Coug Assets & Alumni

KADEN METZGER & TRAVIS TAKUSHI

SEPTEMBER 29TH, 2026

Coug Asset Objectives

- Create a working web application that EECS staff and admin can use
- Streamline the asset sharing process within the EECS department
- Create an efficient and easy to use system

Coug Alumni Objectives

- Create a working web application that EECS staff and admin can use
- Connect Alumni with current EECS staff for opportunities that can be provided for both parties
- Create an efficient and easy to use system

Problem Statement & Scope - Assets

- The EECS department at WSU has assets that are shared within their department. Often times, sharing these assets can be difficult. The current system in place is outdated and inefficient, causing some assets to be purchased that are already present. Our project aims to solve this problem.
- Our target audience is anyone within the WSU EECS department whether they are teachers or students.

Problem Statement & Scope - Alumni

- The EECS department at WSU has a wide alumni base that they can pull many resources from. However, this is not a cohesive network that can easily be accessed by current staff and students. This project should provide a large database that can be queried and provide analytics on the percentage of alumni that want to be connected with and for what purposes.
- Our target audience is anyone within the WSU EECS department whether they are teachers or students.

Technologies Used

- Coug Assets + Alumni is being developed with Next.js and PostgreSQL for our database
- Next.js is a react based framework
- Next.js and react use something known as hooks and states for UI rendering
- Next.js aims to collapse the full stack dev experience into a singular layer for simpler, more efficient development

Functional Requirements- Assets

- User Login
- Asset Search
- Asset Checkout
- Admin Dashboard
- Interfacing with Database
- Routing
- Security

Functional Requirements- Alumni

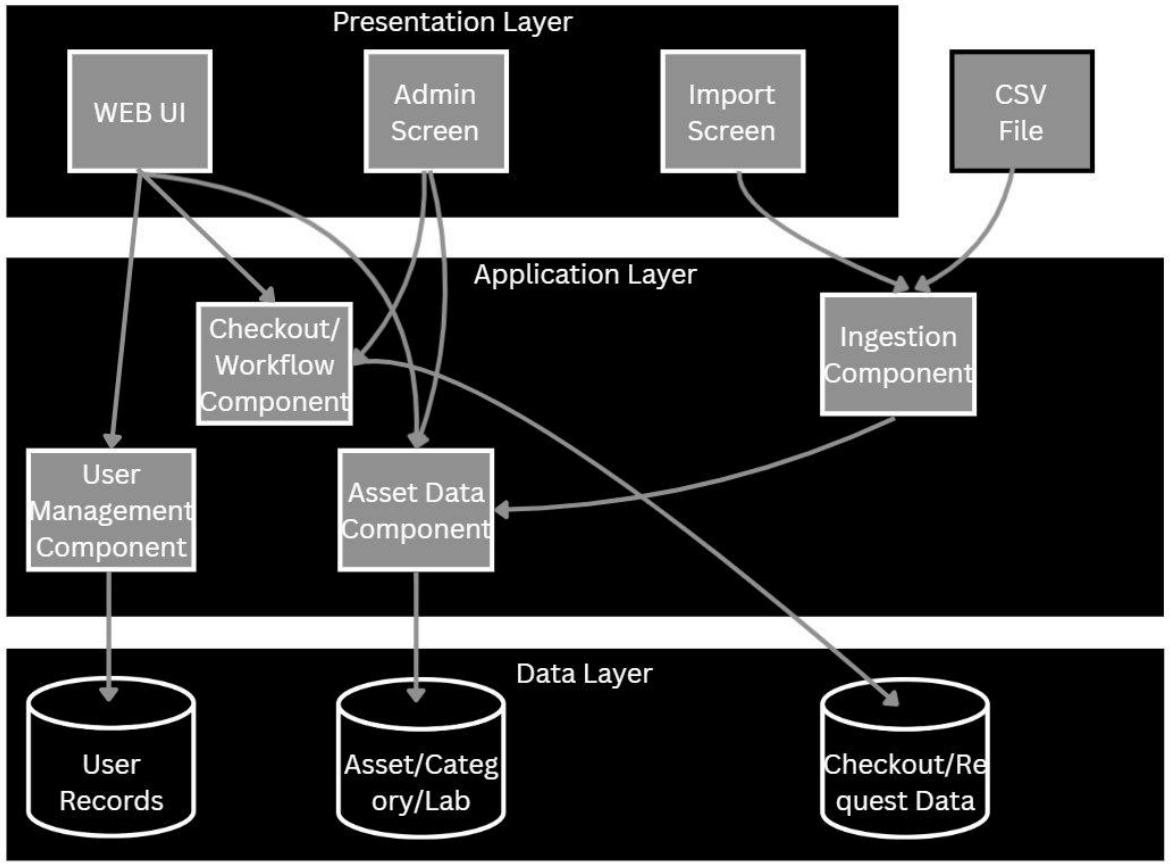
- User Login
- Alumni Search
- View Alumni Records
- Export Data
- Create, Delete, Edit Accounts
- Enforce role based access
- End and Revalidate Sessions

Non-Functional Requirements

- Usability
- Security
- Performance
- Documentation
- Availability
- Maintenance

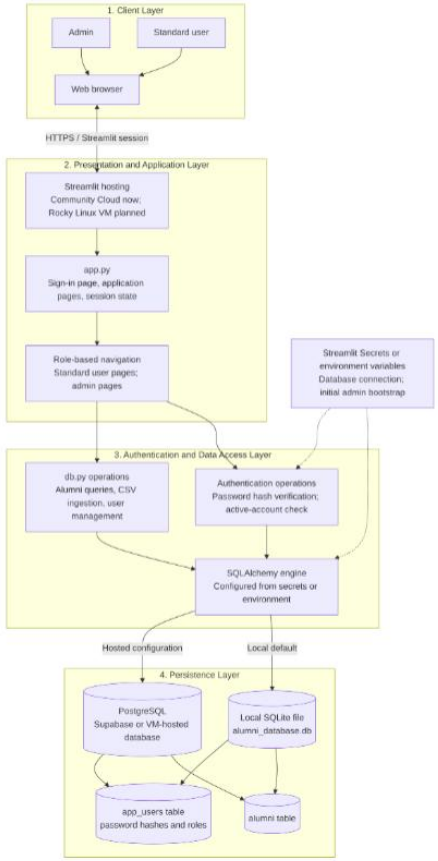
System Architecture - Asset

- UML Diagram



System Architecture - Alumni

- UML Diagram



System Architecture

- Our architecture features 3 layers, UI or presentation layer, application layer, and the database layer
- Our main flow:
  - User interacts with the UI layer
  - UI layer directly runs the application layer software
  - Application layer communicates with the database layer
  - Database layer fetches, updates, removes, etc entries from the database depending on the function called

Sprint 5 Achievements - Assets

- Created barcode scanning capabilities on browser
- Updated upload page to allow for uploading csv of users for quick import
- Automatic Email notifications for overdue items

Sprint 5 Achievements - Alumni

- Hosted live on WSU system
- Categories automatically update on csv import

System demonstration

- Assets
  - Barcode scanning
  - Upload users
  - Automatic email
- Alumni
  - Category Update

Testing

- Unit tests and system test both use vitetest
- Unit tests call route handlers directly
  - docker compose --profile test run --rm test
- System test calls http fetch directly from app
  - docker compose --profile system-test up --build --abort-on-container-exit --exit-code-from system-test system-test

Responsible Use of AI

- Use of chatgpt and github copilot to ask questions when stuck, as well as proper formatting and optimization of systems

Plan and Deliverables for Next Sprint

- Finish report and presentation for end of project

Timeline and Deliverables

- Timeline:
  - Sprint 6: Add final touches, recap functional and non-functional requirements to ensure clients needs are met, and finally deliver a fully working product
- Final Deliverables:
  - A working product that is accessible and intuitive to use
  - Our site must be integrated with OKTA and usable by staff and students
  - Fully fleshed out features that make the site easy to use and manage

Conclusion

- Our product is up to par with client standards so far
- Created a minimum viable product that can be tested with users
- We have made good progress and remain on track to successfully finish the app